

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

FACULTAD DE INGENIERÍA
Ingeniería de Sistemas

Sistema de Gestión Operativa de un Hotel

Hotel Imperial — Sistema de Información

Bases de Datos Avanzadas

Proyecto Final del Semestre · 2026

Integrantes del Equipo

Juan David Amaya Patiño jdamayap@udistrital.edu.co

Juan Pablo Mosquera Marín jpmosqueram@udistrital.edu.co

Código Juan David: 20221020057

Código Juan Pablo: 20221020026

Grupo: 020-82

Docente: René Alejandro Lobo Quintero

Fecha de entrega: Mayo de 2026

Índice

1. Descripción del Sistema	3
1.1. ¿Qué problema resuelve?	3
1.2. Usuarios del Sistema	3
1.3. Funcionalidades Principales	3
2. Casos de Uso	4
3. Diagrama Entidad-Relación	6
3.1. Diagrama ER	6
4. Diccionario de Datos	7
5. Requerimientos Técnicos Implementados	10
5.1. R1 · Base de Datos Relacional	11
5.2. R2 · CRUD Completo	11
5.3. R3 · Procedimiento Almacenado / Función	12
5.4. R4 · Triggers	12
5.5. R5 · Subconsulta	12
5.6. R6 · Vista (VIEW)	13
5.7. R7 · Índices	13
5.8. R8 · Transacciones	13

1. Descripción del Sistema

1.1. ¿Qué problema resuelve?

El sistema gestiona el ciclo operativo completo de un establecimiento de hospedaje, resolviendo principalmente el problema de la **dispersión de datos** al integrar la reserva de habitaciones con el consumo de servicios adicionales. El proceso permite vincular a un cliente específico con una habitación durante un periodo de tiempo determinado, mientras registra de manera simultánea cualquier solicitud de servicio extra que el huésped realice durante su estancia.

Además, el sistema organiza la estructura administrativa al clasificar al personal por áreas y cargos, asignando responsabilidades directas sobre los servicios prestados. En conjunto, el modelo garantiza la **trazabilidad total** del consumo del cliente, desde su registro inicial y datos de contacto hasta la facturación final de los costos asociados a su reserva y los servicios atendidos por los empleados.

La interfaz de interacción es una **aplicación web** conectada a una API REST desarrollada en Spring Boot, con base de datos PostgreSQL.

1.2. Usuarios del Sistema

Tipo de Usuario	Acciones y consultas que realiza
Recepción	Registra y actualiza datos de clientes (teléfonos, correos). Consulta la disponibilidad de habitaciones. Crea y modifica reservas de alojamiento. Consulta los servicios disponibles. Registra solicitudes de servicios adicionales vinculadas a reservas activas, identificándose con su cédula de empleado.
Administración	Gestión total del personal y áreas de trabajo. Configura el catálogo de servicios y sus precios. Consulta la auditoría de cambios sobre empleados.

Cuadro 1: Tipos de usuarios del sistema y sus responsabilidades

1.3. Funcionalidades Principales

- F1.** Registro y consulta de clientes con datos de contacto multivalorados (correos en `CorreoCliente` y teléfonos en `TelefonoCliente`).
- F2.** Gestión del catálogo de habitaciones: creación, consulta por id y actualización de disponibilidad.

- F3.** Creación y modificación de reservas en **Reservar**, vinculando cliente–habitación–fechas, con columna calculada **fecha_rango**.
- F4.** Registro de solicitudes de servicios adicionales en **Solicitar**, asociadas a una reserva activa y al empleado responsable de la atención.
- F5.** Gestión de empleados, áreas de trabajo y catálogo de servicios.
- F6.** Validación automática de solapamiento de reservas para la misma habitación mediante el trigger **trg_no_solapamiento** (BEFORE INSERT OR UPDATE en **Reservar**).
- F7.** Auditoría automática de cambios (INSERT, UPDATE, DELETE) en la tabla **Empleado** mediante el trigger **trg_audit_empleado**, almacenando los valores anteriores y nuevos en formato JSONB.

2. Casos de Uso

A continuación se presentan los casos de uso principales del sistema, en formato de tabla conforme a los lineamientos del proyecto.

CU-01 · Registrar Reserva	
Actor	Recepción
Descripción	El recepcionista selecciona un cliente registrado y una habitación disponible, define fechas de llegada y salida, y crea la reserva mediante POST /reservas . El trigger trg_no_solapamiento valida automáticamente que no existan reservas con fechas superpuestas para la misma habitación antes de confirmar el INSERT.
Precondición	El cliente y la habitación deben estar registrados. La fecha de salida debe ser posterior a la de llegada (CHECK en la tabla Reservar).
Postcondición	La reserva queda registrada en Reservar con su fecha_rango calculado automáticamente.

Cuadro 2: Caso de uso CU-01: Registrar Reserva

CU-02 · Registrar Solicitud de Servicio

Actor	Personal de Servicio
Descripción	El empleado asocia un servicio del catálogo a una reserva activa mediante <code>POST /reservas/solicitar</code> , indicando el nombre de la solicitud, la fecha, la hora y su propia cédula como responsable de la atención.
Precondición	Debe existir una reserva activa (<code>IdReserva</code> válido), el servicio debe estar en el catálogo (<code>IdServicio</code> válido) y el empleado debe estar registrado en <code>Empleado</code> (<code>Cedula</code> válida).
Postcondición	La solicitud queda registrada en <code>Solicitar</code> , vinculada a la reserva, al servicio y al empleado.

Cuadro 3: Caso de uso CU-02: Registrar Solicitud de Servicio

CU-03 · Consultar Disponibilidad de Habitaciones

Actor	Recepción / Personal de Servicio
Descripción	El usuario consulta el catálogo de habitaciones mediante <code>GET /habitaciones</code> o <code>GET /habitaciones/disponibles</code> para verificar cuáles están disponibles, o <code>GET /habitaciones/{id}</code> para el detalle de una habitación específica.
Precondición	Ninguna (consulta de solo lectura).
Postcondición	El sistema retorna la lista de habitaciones o el detalle de una habitación con su estado de <code>Disponibilidad</code> .

Cuadro 4: Caso de uso CU-03: Consultar Disponibilidad de Habitaciones

CU-04 · Registrar Empleado

Actor	Administración
Descripción	El administrador crea un nuevo registro de empleado con sus datos personales, cargo, área asignada y salario mediante <code>POST /empleados</code> . El trigger <code>trg_audit_empleado</code> registra automáticamente en <code>Empleado_audit</code> la operación 'I', el usuario de base de datos, la marca de tiempo y el valor nuevo en formato JSONB.
Precondición	El área referenciada debe existir en <code>Area</code> . La cédula debe tener entre 8 y 10 dígitos numéricos.
Postcondición	El empleado queda registrado y el <code>INSERT</code> queda auditado en <code>Empleado_audit</code> .

Cuadro 5: Caso de uso CU-04: Registrar Empleado

CU-05 · Actualizar Reserva

Actor	Recepción
Descripción	El recepcionista modifica los datos de una reserva existente mediante <code>PUT /reservas/{id}</code> . El trigger <code>trg_no_solapamiento</code> también se activa en el <code>UPDATE</code> para garantizar que las nuevas fechas no generen conflicto con otras reservas.
Precondición	La reserva debe existir (<code>IdReserva</code> válido).
Postcondición	La reserva queda actualizada en <code>Reservar</code> con la nueva <code>fecha_rango</code> calculada.

Cuadro 6: Caso de uso CU-05: Actualizar Reserva

CU-06 · Gestionar Catálogo de Servicios

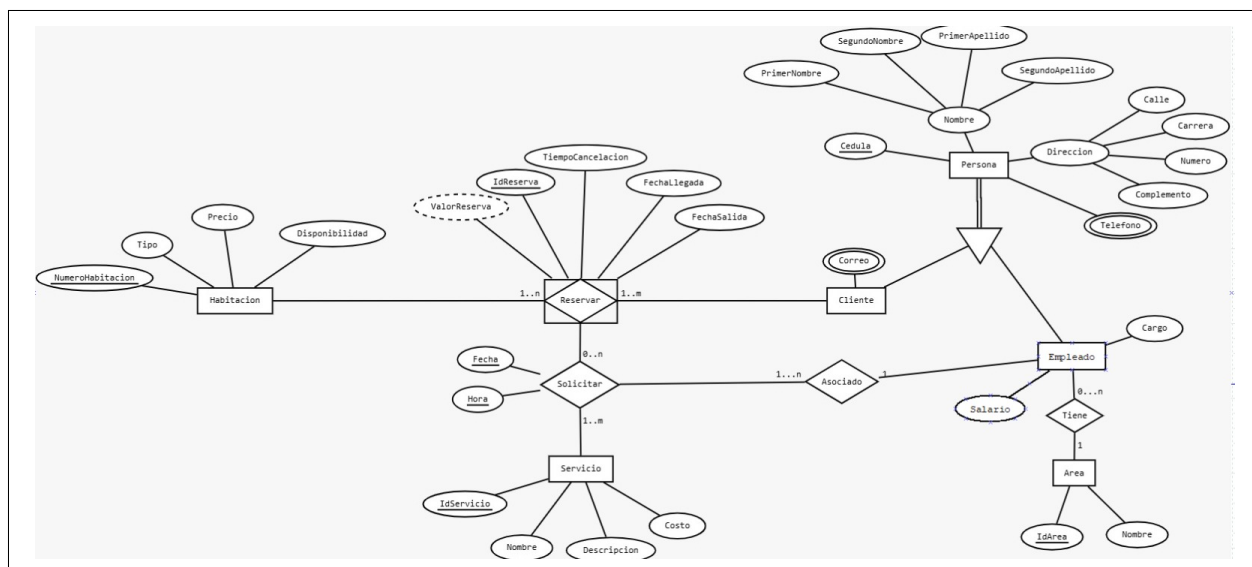
Actor	Administración
Descripción	El administrador crea un nuevo servicio en la tabla <code>Servicio</code> (desayuno, lavandería, etc.) mediante <code>POST /empleados/servicios</code> , definiendo nombre único, descripción y costo.
Precondición	El nombre del servicio no debe existir previamente (restricción <code>UNIQUE</code> en <code>NombreServicio</code>).
Postcondición	El servicio queda registrado y disponible en el catálogo.

Cuadro 7: Caso de uso CU-06: Gestionar Catálogo de Servicios

3. Diagrama Entidad-Relación

El modelo relacional implementado se compone de **diez tablas**, con llaves primarias, foráneas y restricciones `CHECK` / `NOT NULL` relevantes, superando el mínimo de cinco requerido. Requiere la extensión `btree_gist` de PostgreSQL para soportar el tipo `daterange` y el operador de solapamiento `&&`.

3.1. Diagrama ER



Las cardinalidades del modelo son las siguientes:

- **Cliente — Reservar:** 1:N (un cliente puede tener varias reservas).
- **Habitacion — Reservar:** 1:N (una habitación puede tener varias reservas en distintos periodos, sin solapamiento).
- **Reservar — Servicio:** N:M (una reserva puede solicitar varios servicios; un servicio puede aparecer en varias reservas), resuelta con la tabla **Solicitar**.
- **Empleado — Solicitar:** 1:N (un empleado puede atender varias solicitudes; cada solicitud registra la cédula del empleado responsable).
- **Empleado — Area:** N:1 (varios empleados pertenecen a un área).
- **Cliente — TelefonoCliente / CorreoCliente:** 1:N (multivalorado).
- **Empleado — TelefonoEmpleado:** 1:N (multivalorado).

4. Diccionario de Datos

Se describen a continuación las tablas principales del sistema con el nombre, tipo de dato, restricciones y propósito de cada columna.

Tabla: Cliente

Columna	Tipo	Restricción	Propósito
Cedula	varchar(10)	PK, NOT NULL, CHECK (regex)	Número de cédula del cliente. Debe contener entre 8 y 10 dígitos numéricos. Identifica unívocamente al cliente.
PrimerNombre	varchar(255)	NOT NULL	Primer nombre del cliente. Obligatorio para identificación.
SegundoNombre	varchar(255)	—	Segundo nombre del cliente. Opcional.
PrimerApellido	varchar(255)	NOT NULL	Primer apellido del cliente. Obligatorio.
SegundoApellido	varchar(255)	—	Segundo apellido del cliente. Opcional.
Calle	varchar(255)	NOT NULL	Componente de la dirección: nombre o número de la calle.
Carrera	varchar(255)	NOT NULL	Componente de la dirección: número de la carrera.
Numero	varchar(255)	NOT NULL	Número de la propiedad en la dirección.
Complemento	varchar(255)	NOT NULL	Información adicional de la dirección (apto, casa, piso, etc.).

Cuadro 8: Diccionario de datos — tabla **Cliente**

Tabla: Habitacion

Columna	Tipo	Restricción	Propósito
NumeroHabitacion	bigint	PK, NOT NULL, UNIQUE	Número identificador único de la habitación. Clave principal del catálogo.
Tipo	varchar(255)	NOT NULL, CHECK	Categoría de la habitación. Solo acepta Sencilla , Doble o Suite .
Precio	DECIMAL(10,2)	NOT NULL, CHECK (≥ 0)	Precio por noche en la moneda local. No puede ser negativo.
Disponibilidad	boolean	NOT NULL	Indica si la habitación está libre (TRUE) u ocupada (FALSE). Se actualiza mediante PUT /habitaciones/{id} .

Cuadro 9: Diccionario de datos — tabla Habitacion

Tabla: Reservar

Columna	Tipo	Restricción	Propósito
IdReserva	bigint	PK, IDENTITY	Identificador único generado automáticamente por la base de datos.
TiempoCancelacion	bigint	NOT NULL	Plazo máximo (en horas) para cancelar la reserva sin penalidad.
FechaLlegada	date	NOT NULL, CHECK	Fecha de inicio de la estadía. Debe ser anterior a FechaSalida.
FechaSalida	date	NOT NULL, CHECK	Fecha de fin de la estadía. Debe ser posterior a FechaLlegada.
Cedula	varchar(10)	NOT NULL, FK Cliente →	Cédula del cliente que realiza la reserva.
NumeroHabitacion	bigint	NOT NULL, FK → Habitación	Número de la habitación reservada.
fecha_rango	daterange	GENERATE STORED	Columna calculada automáticamente como <code>daterange(FechaLlegada, FechaSalida, '[]')</code> . Usada por el trigger <code>trg_no_solapamiento</code> con el operador <code>&&</code> .

Cuadro 10: Diccionario de datos — tabla Reservar

5. Requerimientos Técnicos Implementados

Esta sección describe cómo el sistema satisface cada requerimiento técnico obligatorio definido en el enunciado del proyecto final.

5.1. R1 · Base de Datos Relacional

El esquema cuenta con **diez tablas** con llaves foráneas y las siguientes restricciones:

- CHECK en Habitación.Tipo: solo permite Sencilla, Doble o Suite.
- CHECK en Habitación.Precio: el precio debe ser ≥ 0 .
- CHECK en Servicio.Costo: el costo debe ser ≥ 0 .
- CHECK en Reservar: FechaSalida > FechaLlegada.
- CHECK (regex) en Empleado.Cedula y Cliente.Cedula: formato numérico de 8 a 10 dígitos ($\sim [0-9]\{8,10\}\$$).
- NOT NULL en campos críticos: cédulas, primer nombre, primer apellido, cargo, salario, fechas de reserva, complemento de dirección.
- UNIQUE en NombreArea y NombreServicio.
- Integridad referencial en todas las llaves foráneas.

5.2. R2 · CRUD Completo

Las entidades Cliente, Habitación, Reservar y Empleado cuentan con los cuatro verbos HTTP implementados en sus controllers y repositories.

Entidad	Método	URL	Descripción
Cliente	GET	/clientes	Listar todos
Cliente	POST	/clientes	Crear cliente
Cliente	PUT	/clientes/{cedula}	Actualizar cliente
Cliente	DELETE	/clientes/{cedula}	Eliminar cliente
Habitación	GET	/habitaciones	Listar todas
Habitación	POST	/habitaciones	Crear habitación
Habitación	PUT	/habitaciones/{id}	Actualizar disponibilidad
Habitación	DELETE	/habitaciones/{id}	Eliminar habitación
Reservar	GET	/reservas	Listar todas
Reservar	POST	/reservas	Crear reserva (con transacción)
Reservar	PUT	/reservas/{id}	Actualizar reserva
Reservar	DELETE	/reservas/{id}	Cancelar reserva (con transacción)
Empleado	GET	/empleados	Listar todos
Empleado	POST	/empleados	Crear empleado
Empleado	PUT	/empleados/{cedula}	Actualizar empleado
Empleado	DELETE	/empleados/{cedula}	Eliminar empleado

Cuadro 11: CRUD completo implementado para las cuatro entidades principales

5.3. R3 · Procedimiento Almacenado / Función

La función `verificar_solapamiento()` encapsula la lógica de detección de solapamiento de fechas. Es invocada automáticamente por el trigger `trg_no_solapamiento` antes de cada `INSERT` o `UPDATE` en `Reservar`. Utiliza el operador `&&` sobre la columna calculada `fecha_rango` (`daterange`) para detectar intersección entre rangos de fechas. Si hay solapamiento, lanza `RAISE EXCEPTION` y aborta la operación.

5.4. R4 · Triggers

Se implementan dos triggers con propósito real:

1. **trg_no_solapamiento**: se dispara `BEFORE INSERT OR UPDATE` en `Reservar` e invoca `verificar_solapamiento()` para garantizar que ninguna habitación tenga dos reservas con fechas superpuestas. Aborta la transacción con `RAISE EXCEPTION` si detecta conflicto. También se activa en `UPDATE`, protegiendo las modificaciones de reservas existentes.
2. **trg_audit_empleado**: se dispara `AFTER INSERT OR UPDATE OR DELETE` en `Empleado` e invoca `fn_audit_empleado()` para registrar en `Empleado_audit` el tipo de operación ('I', 'U' o 'D'), el usuario de base de datos (`current_user`), la marca de tiempo y los valores anteriores (`dato_viejo`) y nuevos (`dato_nuevo`) en formato JSONB mediante `row_to_json`.

5.5. R5 · Subconsulta

Se implementa la subconsulta correlacionadas en `ClienteRepository`. La consulta principal retorna los clientes que **no** tienen ninguna reserva activa, usando `NOT EXISTS` sobre una subconsulta interna que detecta reservas vigentes (aquellas cuya `FechaSalida` es mayor o igual a la fecha actual).

La subconsulta es **correlacionada**: por cada fila de `Cliente` en la consulta externa, la subconsulta interna se evalúa filtrando únicamente las reservas que pertenecen a ese cliente (`r.Cedula = c.Cedula`) y que aún no han concluido (`r.FechaSalida >= CURRENT_DATE`). Si dicha subconsulta no devuelve ninguna fila, `NOT EXISTS` se evalúa como verdadero y el cliente es incluido en el resultado.

Implementación en la capa de acceso a datos

El método se agrega en `ClienteRepository` reutilizando el `clienteMapper` existente, sin requerir nuevo mapeo:

El endpoint correspondiente se expone en `ClienteController` mediante:

Una petición `GET /clientes/sin-reserva-activa` devuelve la lista de clientes registrados que actualmente no poseen ninguna reserva vigente en el sistema.

5.6. R6 · Vista (VIEW)

La vista `Vista_Cliente_Reserva_Servicios` consolida en una sola proyección los datos de `Cliente`, `Reservar`, `Solicitar` y `Servicio`. Es consumida desde la API mediante el método `findAllClienteReservaServicios()` en `ClienteRepository`, que ejecuta `SELECT ... FROM vista_cliente_reserva_servicios` y mapea el resultado al modelo `ClienteReservaServicioView`. El endpoint `GET /clientes/reserva-servicios` en `ClienteController` la expone al frontend.

5.7. R7 · Índices

Se crean cuatro índices en el script de migración sobre columnas consultadas frecuentemente:

Índice	Justificación
<code>idx_reservar_cedula</code>	La columna <code>Cedula</code> en <code>Reservar</code> se filtra para consultar el historial de reservas de un cliente específico.
<code>idx_reservar_habitacion</code>	La columna <code>NumeroHabitacion</code> en <code>Reservar</code> se evalúa en cada ejecución del trigger <code>trg_no_solapamiento</code> .
<code>idx_habitacion_disponibilidad</code>	Se filtra por <code>Disponibilidad = TRUE</code> en el endpoint <code>GET /habitaciones/disponibles</code> ; el índice acelera este filtro booleano.
<code>idx_solicitar_idreserva</code>	La columna <code>IdReserva</code> en <code>Solicitar</code> es usada en el <code>JOIN</code> de la vista y en el <code>DELETE FROM Solicitar</code> dentro de <code>deleteById()</code> de <code>ReservarRepository</code> .

Cuadro 12: Índices implementados y su justificación

5.8. R8 · Transacciones

El método `save()` de `ReservarRepository` implementa una transacción explícita con nivel de aislamiento `SERIALIZABLE`: obtiene una conexión directa vía `getDataSource().getConnection()`, desactiva el autocommit, ejecuta `BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE`, realiza el `INSERT` en `Reservar` y finaliza con `COMMIT`. Si el trigger `trg_no_solapamiento` lanza una excepción o cualquier otra operación falla, el bloque `catch` ejecuta `ROLLBACK` para que ningún cambio persista.

El método `deleteById()` aplica el mismo patrón: dentro de la misma transacción elimina primero las filas dependientes de `Solicitar` (para respetar la FK) y luego el registro de `Reservar`, garantizando atomicidad en la cancelación.